CS-338 Intro to the Theory of Computation

- Lecture #14
- **Complexity Theory: NP Completeness**

- Prof Pietro S. Oliveto
  Department of Computer Science and Engineering
  Southern University of Science and Technology (SUSTech)
- olivetop@sustech.edu.cn
  https://faculty.sustech.edu.cn/olivetop
- Office: Room 113

Overview

Last time:

- NTIME$\big(t(n)\big)$, NP
- $HAMPATH, COMPOSITES, CLIQUE, SUBSET-SUM \in NP$
- $COMPOSITES, PRIMES \in P$
- $PRIMES \in coNP$
- P vs NP problem

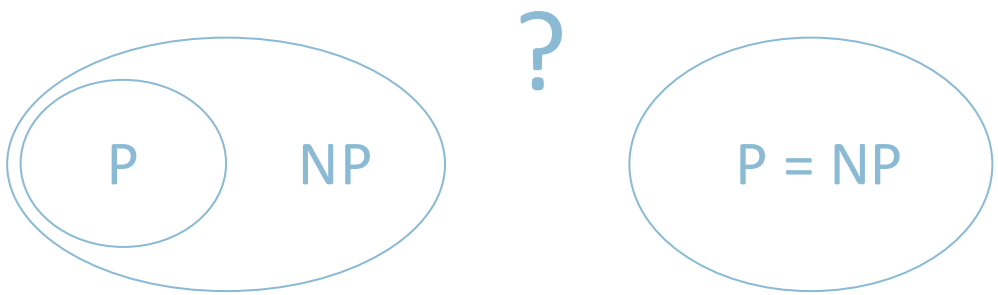
Today: (Sipser §7.5)

- Polynomial-time reducibility
- NP-completeness

Quick Review

NP = All languages where can <u>verify</u> membership quickly
P = All languages where can <u>test</u> membership quickly

P versus NP question: Does P = NP?



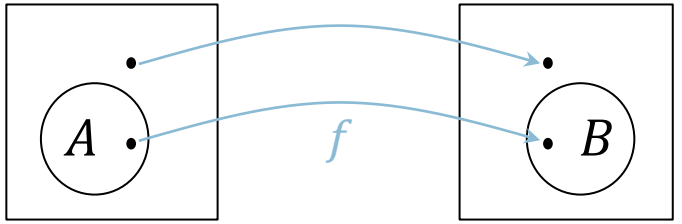
$SAT = \{\langle\phi\rangle | \ \phi$ is a satisfiable Boolean formula$\}$

Cook-Levin Theorem: $SAT \in$ P $\rightarrow$ P = NP
Proof plan: Show that every $A \in$ NP is polynomial time reducible to SAT.

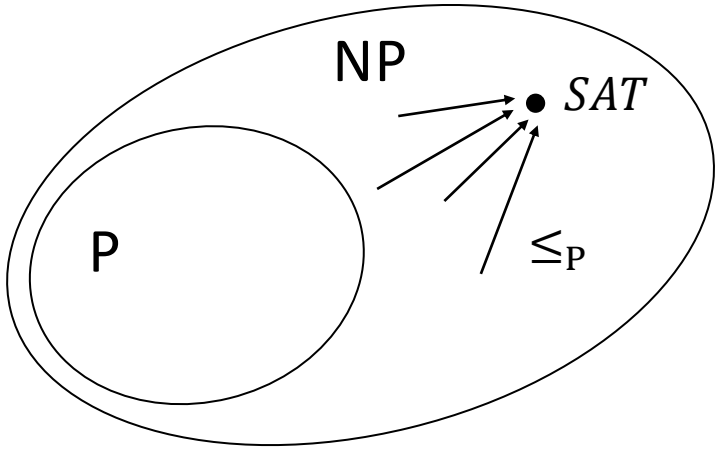
Polynomial Time Reducibility

Defn: A is <u>polynomial time reducible</u> to B ($A \leq_{\mathrm{P}} B$) if $A \leq_{\mathrm{m}} B$ by a reduction function that is computable in polynomial time.

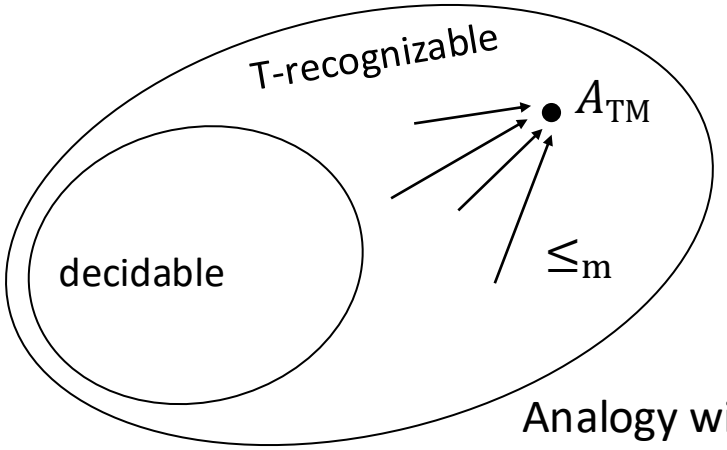
Theorem: If $A \leq_{\mathrm{P}} B$ and $B \in \mathrm{P}$ then $A \in \mathrm{P}$.



f is computable in polynomial time



Idea to show $SAT \in \mathrm{P} \rightarrow \mathrm{P} = \mathrm{NP}$



Analogy with A_{TM}

$\leq_P$ Example: $3SAT$ and $CLIQUE$

Defn: A Boolean formula ϕ is in Conjunctive Normal Form (CNF) if it has the form $\phi = (x \vee \overline{y} \vee z) \wedge (\overline{x} \vee \overline{s} \vee z \vee u) \wedge \cdots \wedge (\overline{z} \vee \overline{u})$

clause clause literals

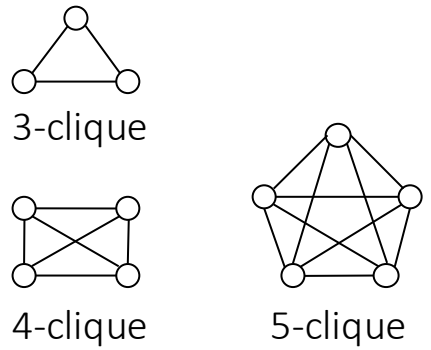
Literal: a variable or a negated variable
Clause: an OR ($\vee$) of literals.
CNF: an AND ($\wedge$) of clauses.
3CNF: a CNF with exactly 3 literals in each clause.

$3SAT = \{\langle\phi\rangle | \ \phi \text{ is a satisfiable 3CNF formula}\}$

Defn: A k-clique in a graph is a subset of k nodes all directly connected by edges.

$CLIQUE = \{\langle G, k\rangle | \text{ graph } G \text{ contains a } k\text{-clique}\}$

Will show: $3SAT \leq_P CLIQUE$



$3SAT \leq_P CLIQUE$

Theorem: $3SAT \leq_P CLIQUE$

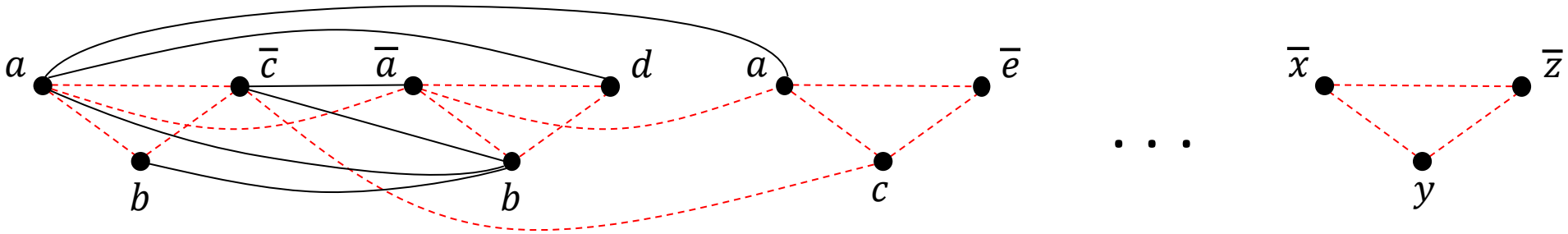
Proof: Give polynomial-time reduction f that maps ϕ to G, k where ϕ is satisfiable iff G has a k-clique.

A satisfying assignment to a CNF formula has ≥1 true literal in each clause.

$$\phi = (a \vee b \vee \overline{c}) \wedge (\overline{a} \vee b \vee d) \wedge (a \vee c \vee \overline{e}) \wedge \cdots \wedge (\overline{x} \vee y \vee \overline{z})$$

f ↓

G =



k = # clauses

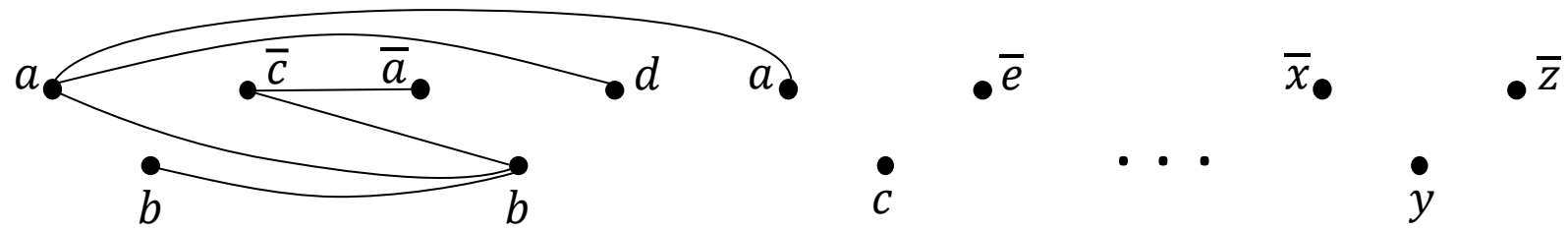
Forbidden edges:
1) within a clause
2) inconsistent labels (a and $\overline{a}$)

G has <u>all</u> non-forbidden edges

$3SAT \leq_P CLIQUE$ conclusion

$$\phi = (a \vee b \vee \overline{c}) \wedge (\overline{a} \vee b \vee d) \wedge (a \vee c \vee \overline{e}) \wedge \cdots \wedge (\overline{x} \vee y \vee \overline{z})$$

$f \downarrow$

G = 

k = # clauses

Claim: ϕ is satisfiable iff G has a k-clique

(→) Take any satisfying assignment to ϕ. Pick 1 true literal in each clause.
The corresponding nodes in G are a k-clique because they don't have forbidden edges.

(←) Take any k-clique in G. It must have 1 node in each clause.
Set each corresponding literal TRUE. That gives a satisfying assignment to ϕ.

The reduction f is computable in polynomial time.

Corollary: $CLIQUE \in \mathrm{P} \rightarrow 3SAT \in \mathrm{P}$

Check-in 15.1

Does this proof require 3 literals per clause?

(a) Yes, to prove the claim.

(b) Yes, to show it is in poly time.

(c) No, it works for any size clauses.

Check-in 15.1

NP-completeness

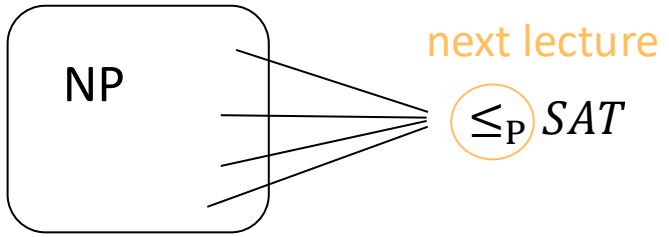
Defn: B is NP-complete if

1) $B \in$ NP
2) For all $A \in$ NP, $A \leq_{\mathrm{P}} B$

If B is NP-complete and $B \in$ P then P = NP.

Cook-Levin Theorem: SAT is NP-complete

Proof: Next lecture; assume true



To show some language C is NP-complete, show $3SAT \leq_P C$.

or some other previously shown NP-complete language

Check-in 15.2

What language that we've previously seen is most analogous to SAT?

(a) A_{TM}

(b) E_{TM}

(c) $\{0^k 1^k \mid k \geq 0\}$

Check-in 15.2

$HAMPATH$ is NP-complete

Theorem: $HAMPATH$ is NP-complete

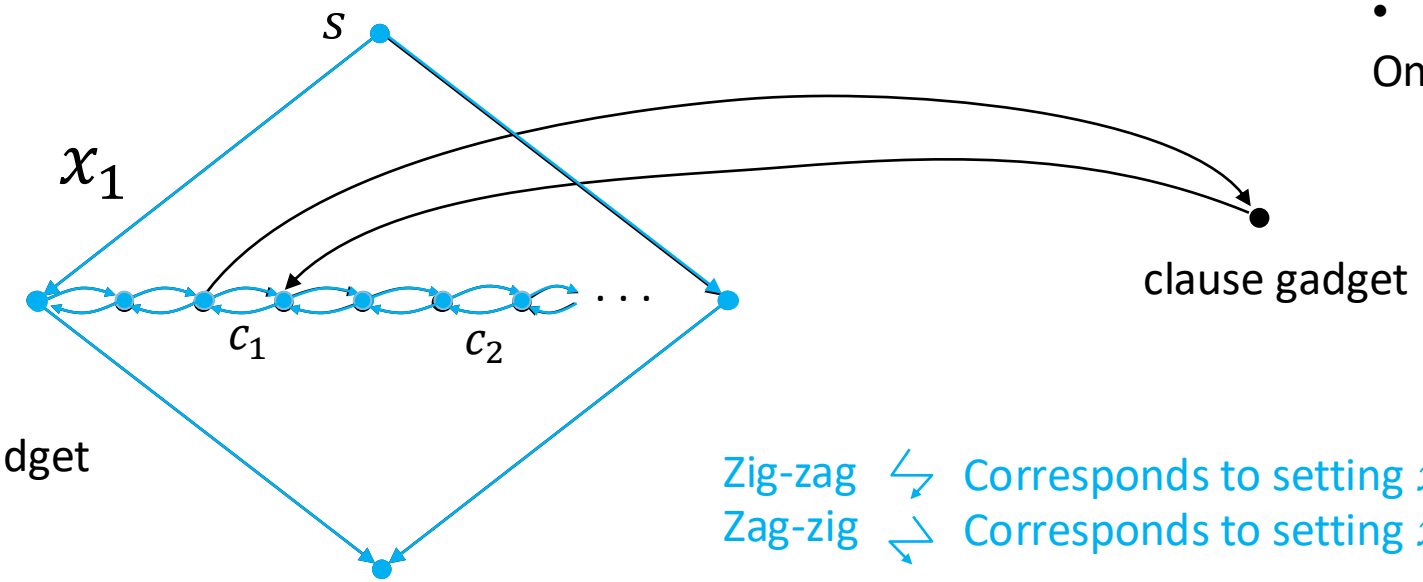
Proof: Show $3SAT \leq_P HAMPATH$ (assumes $3SAT$ is NP-complete)

Idea: "Simulate" variables and clauses with "gadgets"

$$\phi = (x_1 \vee \overline{x}_2 \vee x_3) \wedge (\overline{x}_1 \vee x_2 \vee x_4) \wedge \cdots \wedge (\qquad)$$

$f \downarrow$

$\langle G, s, t \rangle$



variable gadget

For each variable gadget:

- Let k be the number of clauses
- Horizontal row contains $3k + 1$ nodes + the two endpoints
- one pair for each clause + a separator

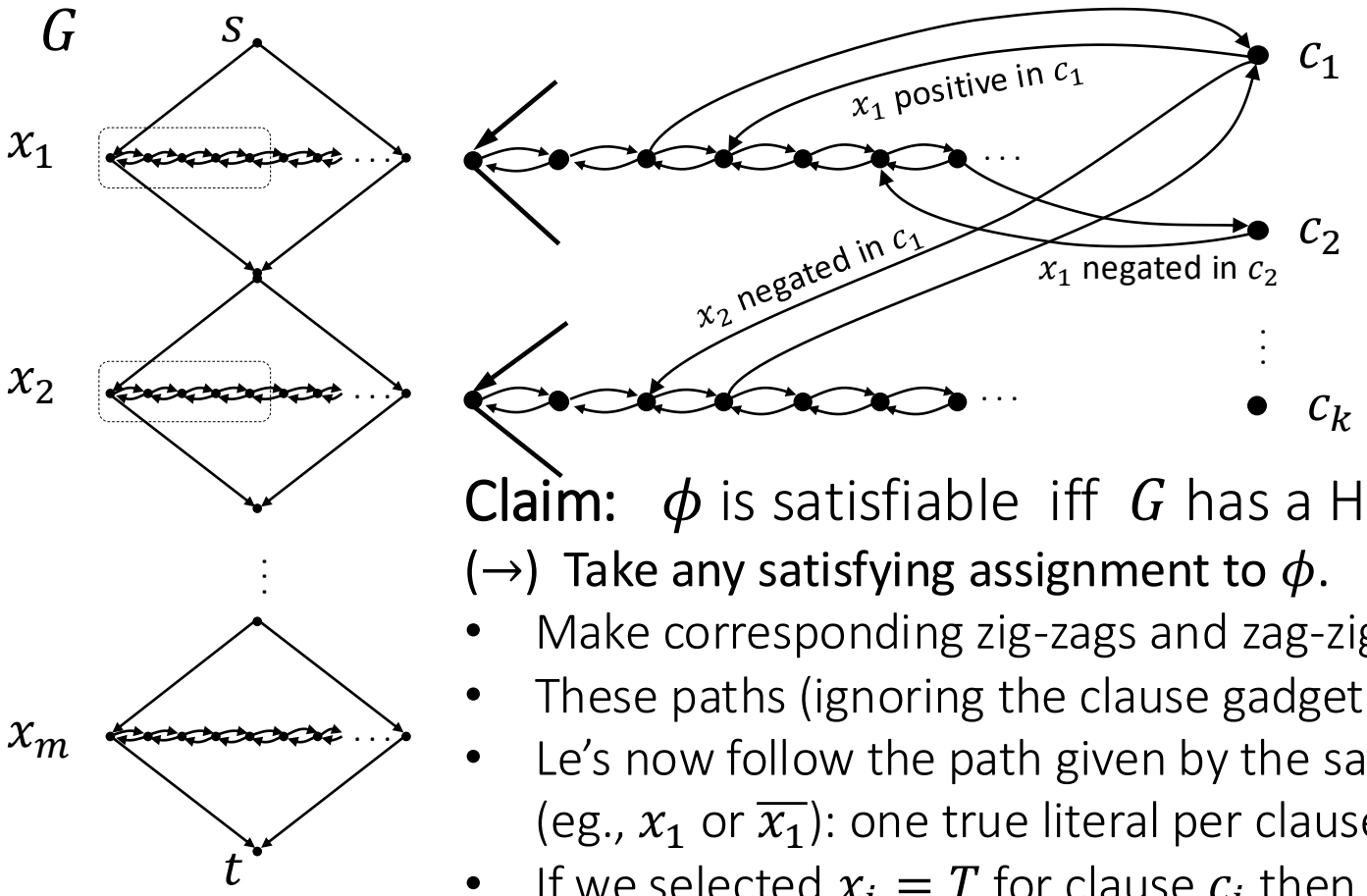
One clause gadget per clause (one node)

Zig-zag ↯ Corresponds to setting x_1 TRUE

Zag-zig ↯ Corresponds to setting x_1 FALSE

Construction of G

$$\phi = \underbrace{(x_1 \vee \overline{x}_2 \vee x_3)}_{c_1} \wedge \underbrace{(\overline{x}_1 \vee x_2 \vee x_4)}_{c_2} \wedge \cdots \wedge \underbrace{(\quad x_m \quad)}_{c_k}$$



m variables
k clauses

The reduction f is computable in polynomial time.

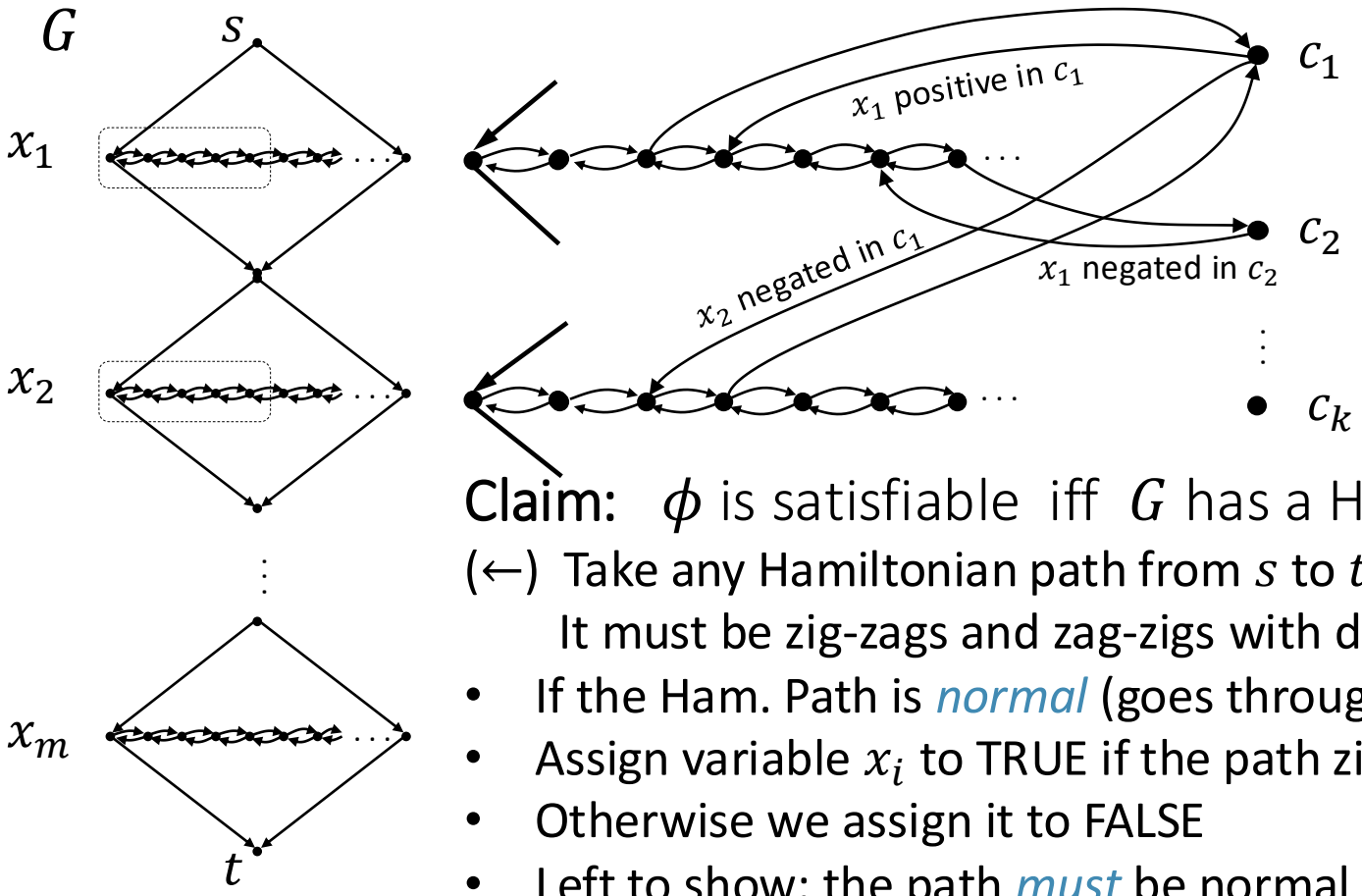
Claim: ϕ is satisfiable iff G has a Hamiltonian path from s to t.

(→) Take any satisfying assignment to ϕ.

- Make corresponding zig-zags and zag-zigs through variable gadgets from s to t.
- These paths (ignoring the clause gadgets) all would be Hamiltonian (there are many)
- Le's now follow the path given by the satisfying assignment (selecting the true literals (eg., x_1 or $\overline{x_1}$): one true literal per clause
- If we selected $x_i = T$ for clause c_j then we can detour through c_j with a zig-zag
- Otherwise ($\overline{x_1} = \mathrm{T}$) we can detour through c_j with a zag-zig
- If several literals in a clause are True, only one option is taken

Construction of G

$$\phi = \underbrace{(x_1 \vee \overline{x}_2 \vee x_3)}_{c_1} \wedge \underbrace{(\overline{x}_1 \vee x_2 \vee x_4)}_{c_2} \wedge \cdots \wedge \underbrace{(\quad x_m \quad)}_{c_k}$$



m variables
k clauses

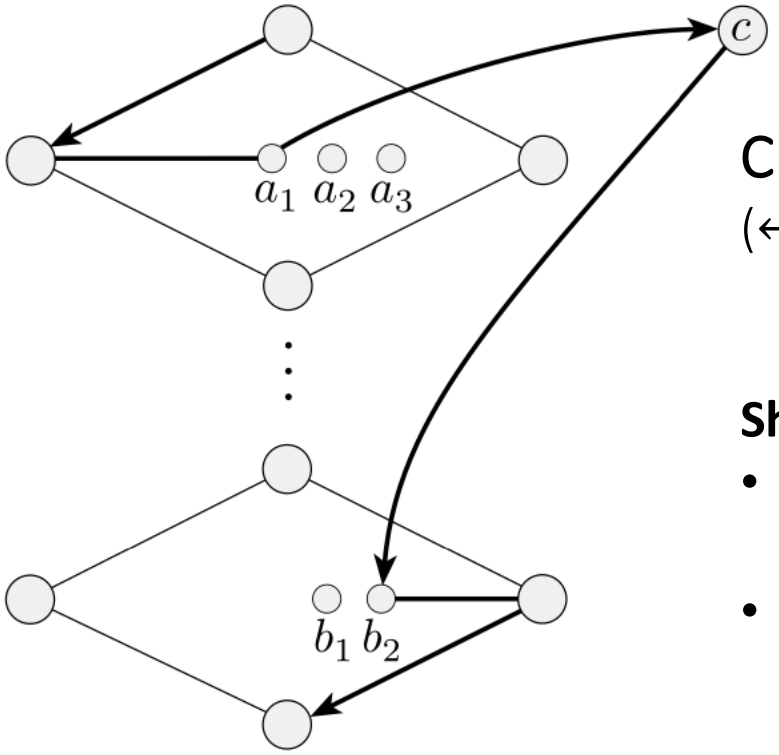
Claim: ϕ is satisfiable iff G has a Hamiltonian path from s to t.

($\leftarrow$) Take any Hamiltonian path from s to t.

It must be zig-zags and zag-zigs with detours to visit all c_i.

- If the Ham. Path is *normal* (goes through the diamonds in order) then it's easy
- Assign variable x_i to TRUE if the path zig-zags through the diamond
- Otherwise we assign it to FALSE
- Left to show: the path *must* be normal

Hamiltonian construction path must be normal



Claim: ϕ is satisfiable iff G has a Hamiltonian path from s to t.

(←) Take any Hamiltonian path from s to t.

Show it must be **zig-zags and zag-zigs** with detours to visit all c_i.

Get corresponding truth asst. It must satisfy ϕ because path visits all c_i.

Show the path is normal

- It fails normality if it enters a clause from one diamond (a_1) and exits into another diamond (b_2)
- Then either a_2 or a_3 is a separator node
  - **a_2 separator**: the only edges entering a_2 are a_1 and a_3: you can't get to a_2 without hitting a_1 again => Path not Hamiltonian
  - **a_3 separator**: then a_1 and a_2 are the same clause pair: you can't get to a_2 without going through a_1 or c
- Thus the path must be normal

Quick review of today

1. NP-completeness
2. SAT and $3SAT$
3. $3SAT \leq_P HAMPATH$
4. $3SAT \leq_P CLIQUE$
5. Strategy for proving NP-completeness:
   Reduce from $3SAT$ by constructing gadgets
   that simulate variables and clauses.